

Reading TypeScript: Functions, Async, Symbols

Three decoders for code you have never seen — every statement with how to say it out loud

October 2026

How to use the “Read aloud” column.

It is the sentence you say in your head (or on the Loom) when you meet that line. Say it exactly; if the sentence does not fit the line you are looking at, the line is doing something else and you have found a place to slow down.

1. Reading a function

The shape

```
export async function getUser(id: string, retries = 3, opts?: Opts): Promise<User | null> {  
  //  
  //  
  //  
}
```

export async keyword name required default optional return type (Promise: async)

Read aloud: “Exported async function `getUser` that takes a required string `id`, a number `retries` defaulting to 3, and an optional `opts`, and promises to return a `User` or `null`.”

Then, in this order: find every `return` in the body and ask whether each one matches that promise.

Every function shape

Written	Read aloud	What to check
<code>function f(x: number): string { ... }</code>	"function <code>f</code> takes a number <code>x</code> , returns a string"	every path returns a string?
<code>const f = (x: number): string => { ... }</code>	" <code>f</code> is an arrow function: takes a number <code>x</code> , returns a string, body in braces"	braces need an explicit <code>return</code>
<code>const f = (x: number) => x * 2</code>	" <code>f</code> is an arrow: takes <code>x</code> , returns <code>x</code> times 2"	no braces, so the expression IS the return
<code>const f = function (x: number) { ... }</code>	" <code>f</code> is a function expression taking <code>x</code> "	old style; <code>this</code> depends on caller
<code>async function f(): Promise<T></code>	"async function <code>f</code> , returns a promise of <code>T</code> "	callers must <code>await</code> it
<code>const f = async (x) => { ... }</code>	" <code>f</code> is an async arrow taking <code>x</code> "	returns a Promise whatever the body returns
<code>f(x: number) { ... }</code> inside <code>class</code>	"method <code>f</code> on the class, takes <code>x</code> "	<code>this</code> is the instance only when called as <code>obj.f()</code>
<code>static f() { ... }</code>	"static method <code>f</code> , called on the class itself"	<code>A.f()</code> , not <code>new A().f()</code>
<code>get total() { ... }</code>	"getter <code>total</code> : computed when you read <code>obj.total</code> "	no parentheses at the call site
<code>function f<T>(x: T): T</code>	"generic <code>f</code> : takes some type <code>T</code> , returns the same <code>T</code> "	<code>T</code> is a placeholder; skip it
<code>function f(x: unknown): x is Id</code>	"type guard: returns true if <code>x</code> is an <code>Id</code> , and tells the compiler so"	is the body's check actually correct?
<code>function f(msg: string): never</code>	" <code>f</code> never returns normally"	must throw or loop
<code>function f(): void</code>	" <code>f</code> returns nothing useful"	a <code>return value</code> here is a type error
<code>(x: number) => string</code> in a TYPE position	"a function type: number in, string out"	describes a callback parameter, not a function body
<code>arr.map((x) => ...)</code>	"map over <code>arr</code> , calling this arrow once per element"	the arrow is an argument, not a declaration
<code>function f(a: string): void; function f(a: number): void; function f(a: any) {}</code>	"two overload signatures, then one implementation"	the last one is the real body

Parameters

Written	Read aloud	Inside the body
<code>x: number</code>	"required number <code>x</code> "	<code>x</code> is a number

Written	Read aloud	Inside the body
<code>x?: number</code>	"optional <code>x</code> , number if present"	<code>x</code> is <code>number</code> <code>undefined</code> ; check before use
<code>x = 5</code>	" <code>x</code> defaults to 5 when not passed"	default applies only for <code>undefined</code> , not <code>null</code>
<code>x: number = 5</code>	"number <code>x</code> , default 5"	same
<code>...rest: number[]</code>	"collect any further arguments into the array <code>rest</code> "	<code>rest</code> is always an array, possibly empty
<code>{ a, b }: Opts</code>	"destructure the <code>Opts</code> argument into locals <code>a</code> and <code>b</code> "	caller passes one object <code>{ a: 1, b: 2 }</code>
<code>{ a = 1, b }: Opts = {}</code>	"destructure <code>Opts</code> , <code>a</code> defaults to 1, and the whole argument defaults to an empty object"	caller may pass nothing at all
<code>[x, y]: [number, number]</code>	"destructure a two-number tuple into <code>x</code> and <code>y</code> "	
<code>this: Window</code> as first param	"declares what <code>this</code> must be"	not a real argument; callers skip it

What does it actually return? Trace every path

```

function find(xs: User[], id: string): User { // "find takes users and an id, returns a User"
  for (const u of xs) { // "for each u in xs"
    if (u.id === id) return u; // "if u's id equals id, return u" <- path
  }
} // "end of loop; nothing returned" <- path

1: a User
2: undefined x

```

Read aloud: "Declared to return a `User`, but if no id matches, the function falls off the end and returns `undefined`. The declaration lies to the caller."

Pattern	Read aloud	Returns
no <code>return</code> at all	"this function returns nothing"	<code>undefined</code>
<code>return;</code>	"return with no value"	<code>undefined</code>
<code>if (c) return a;</code> with no else	"returns <code>a</code> only when <code>c</code> ; otherwise falls through"	<code>a</code> or <code>undefined</code>
<code>x => { x * 2 }</code>	"arrow with a block body and no return"	<code>undefined</code> ×
<code>x => x * 2</code>	"arrow that returns <code>x</code> times 2"	<code>x * 2</code>
<code>async</code> function with <code>return 5</code>	"async, so this resolves to 5 inside a Promise"	<code>Promise<number></code>
<code>async</code> function that throws	"async, so the throw becomes a rejected Promise"	rejected Promise
<code>return await p</code>	"wait for <code>p</code> , then return its value"	caught by this function's <code>try</code>
<code>return p</code>	"hand the Promise <code>p</code> straight back"	NOT caught by this function's <code>try</code>
<code>arr.forEach(cb)</code>	"forEach always returns undefined"	<code>undefined</code>
<code>arr.map(x => { x * 2 })</code>	"map with a block body and no return"	array of <code>undefined</code> ×

Trace one call

- Write the arguments: `getUser("42")` → "`id` is the string 42, `retries` is 3, `opts` is undefined."
- Walk each line saying what it does and what each variable holds afterwards.
- At each `if`, say which branch your values take, and what the other branch would need.
- At each `return`, say "returns X, declared Y; match?"
- Repeat with an edge input: `"", undefined, 0`, an id that does not exist.

Review questions for any function

- Does every `return` path match the declared type? (a missing path returns `undefined`)
- Can any parameter be `undefined` where the body assumes it is not?
- Is every `?` or `= default` parameter handled when the caller omits it?
- Does it mutate a parameter (`xs.sort()`, `opts.x = ...`) the caller still uses?
- If `async`: does every caller `await` it?

2. Reading an async block

The annotated example

```
async function load(ids: string[]): Promise<User[]> {  
  (sync)  const cache = getCache();           // 1. "cache is whatever getCache returns, right now"  
  string) const token = await getToken();     // 2. "WAIT for getToken, then token is its value" (a  
  started" const p = fetchUser(ids[0]);       // 3. "p is the PROMISE from fetchUser; the request has  
  const u = await p;                         // 4. "WAIT for p, then u is the User"  
  
  const all = await Promise.all(             // 5. "start every fetch at once, WAIT for all of them"  
    ids.map((id) => fetchUser(id))           // "map each id to a Promise<User>"  
  );                                         // "all is the array of Users, in input order"  
  
  ids.forEach(async (id) => {                // 6. "for each id, start an async callback" x  
    for it" await save(id);                  // "...which waits for save, but forEach does not wait"  
  });                                       // "so load() continues before any save finishes"  
  
  try {  
    const r = fetchUser("x");               // 7. "r is a Promise, not awaited" x  
    return [u, ...all];                     // "a rejection of r escapes this try"  
  } catch (e) {                             // 8. "e is unknown"  
    throw new Error("load failed", { cause: e }); // "wrap and rethrow, keeping the cause"  
  }  
}
```

Promise or value? Decide for every line

Line looks like	Read aloud	Is this a bug?
<code>const x = await f()</code>	"wait for f , x is its value"	✓
<code>const x = f()</code> where f is async	" x is the Promise from f , not the value"	✗ unless awaited later or passed to <code>Promise.all</code>
<code>const x = f().then(g)</code>	" x is a Promise of g applied to f 's result"	✓ if awaited or returned
<code>return f()</code> inside async	"hand f 's Promise back to my caller"	✓, but not caught by this function's <code>try</code>
<code>return await f()</code>	"wait for f , then return the value"	✓, and caught by this function's <code>try</code>
<code>f();</code> alone, f async	"start f and walk away"	✗ unless <code>.catch</code> is attached or intended
<code>await f(); await g();</code>	"wait for f , then wait for g " (sequential)	✓ if g needs f 's result; slow if independent
<code>await Promise.all([f(), g()])</code>	"start f and g together, wait for both, results in order"	✓; rejects on the FIRST failure
<code>await Promise.allSettled([...])</code>	"start all, wait for all, report each as fulfilled or rejected"	✓ when partial failure is acceptable
<code>arr.map(async x => ...)</code>	"map each x to a Promise"	✗ until wrapped in <code>Promise.all</code>
<code>arr.forEach(async x => ...)</code>	"start a callback per x and do not wait for any"	✗ almost always
<code>for (const x of arr) await f(x)</code>	"for each x , wait for f before the next"	✓ if order matters; otherwise slow
<code>await</code> inside a non-async function	"syntax error"	
<code>res.json()</code>	"a Promise of the parsed body"	✗ if used without <code>await</code>
<code>JSON.parse(s)</code>	"parse s now, synchronously"	throws on bad input; needs <code>try</code>

Where does an error go?

Situation	Read aloud	Where the error lands
<code>throw</code> inside an async function	"the Promise this function returned rejects"	caller's <code>await</code>
<code>await p</code> where p rejects	"the rejection is thrown right here at the await"	enclosing <code>try</code>
p not awaited, rejects	"nobody is listening"	unhandled rejection; crashes Node

Situation	Read aloud	Where the error lands
<code>try { f() } catch {}</code> with <code>async f</code>	"the try finishes before <code>f</code> 's Promise settles"	NOT caught
<code>try { await f() } catch {}</code>	"the try waits, so a rejection is thrown inside it"	caught
<code>try { return f() } catch {}</code>	"the Promise leaves the try unresolved"	NOT caught
<code>try { return await f() } catch {}</code>	"waits inside the try"	caught
<code>Promise.all</code> with one rejection	"the first failure rejects the whole thing; the rest are ignored"	enclosing <code>try</code>
<code>.catch(() => {})</code>	"swallow the error silently"	nowhere; a finding unless intended
<code>catch (e) { e.message }</code>	" <code>e</code> is unknown; <code>.message</code> is a type error"	needs <code>instanceof Error</code>

Shared state across an `await`

Every `await` is a point where another request can run. Read aloud for any shared variable: “*read before the await, write after the await: someone else can run in between.*”

```
const n = counter.get(k) ?? 0;    // "read n"
await db.save(...);              // "WAIT; another request may read the same n here"
counter.set(k, n + 1);           // "write n plus 1; the other request's increment is lost"
```

Cache version: `if (!cache.has(k)) cache.set(k, await load(k))` reads “*if not cached, wait for load, then store*”; every concurrent miss passes the `if` before any store happens. Fix: store the Promise, `cache.set(k, load(k))`, then `await cache.get(k)`.

Timers

Written	Read aloud	Trap
<code>setTimeout(fn, ms)</code>	"run <code>fn</code> once, after <code>ms</code> milliseconds"	<code>clearTimeout(id)</code> cancels
<code>setInterval(fn, ms)</code>	"run <code>fn</code> every <code>ms</code> milliseconds, forever"	never cleared = leak; <code>.unref()</code> lets Node exit
<code>await new Promise(r => setTimeout(r, ms))</code>	"sleep for <code>ms</code> "	the Python <code>await asyncio.sleep</code>
<code>setTimeout(fn, 0)</code>	"run <code>fn</code> after everything pending"	still after every <code>.then</code>
<code>Date.now()</code>	"milliseconds since 1970, as a number"	hard-coded = untestable; inject a clock

Review questions for any async block

- For each line: Promise or value? Is every Promise awaited, returned, or passed to `Promise.all`?
- Is there an `async` callback inside `forEach`?
- Is there a `try` whose body has an un-awaited call?
- Independent awaits in sequence that could be parallel?
- Shared state read before an `await` and written after?
- `res.ok` checked before `res.json()`?
- Interval cleared or `.unref()`'d?

3. Symbol decoder

The same character means different things by position. Find the position first, then say the sentence.

?

Position	Read aloud	Example
after a property name in a type	" a , optional"	<code>{ a?: number }</code>
after a parameter name	" x , optional"	<code>f(x?: string)</code>
between two expressions, with a : later	"if ... then ... else ..."	<code>ok ? a : b</code>
before . , [or (	" o , if it exists, dot a "	<code>o?.a</code> , <code>arr?.[0]</code> , <code>cb?.()</code>
??	" x , or if missing, 10"	<code>x ?? 10</code>
??=	"set x to 10 if it is missing"	<code>x ??= 10</code>

!

Position	Read aloud	Example
before an expression	"not x "	<code>!x</code> , <code>!arr.length</code>
after an expression	" user , which I promise is not null" (compile-time only)	<code>user!.name</code>
!==	"is not strictly equal to"	<code>a !== b</code>
!=	"is not loosely equal to" (flag it)	<code>a != b</code>

:

Position	Read aloud	Example
after a name in a declaration or parameter	" x , of type number"	<code>x: number</code>
after) of a function	"returns a string"	<code>f(): string</code>
inside { } in a VALUE	"key a holds 1"	<code>{ a: 1 }</code>
inside { } in a TYPE	"key a is a number"	<code>{ a: number }</code>
in destructuring	"take a out and call it x "	<code>const { a: x } = o</code>
second part of a ternary	"else"	<code>c ? a : b</code>
after case	"in the case of"	<code>case "x":</code>

=, ==, ===, =>

Written	Read aloud
=	"set ... to ..." (never "equals")
==	"loosely equals, after converting" (flag)
===	"strictly equals"
=>	"returns" / "maps to"
>= , <=	"at least" / "at most"; check the boundary value
+= , -= , *=	"add to", "subtract from", "multiply by"
 = , &&= , ??=	"set if falsy", "set if truthy", "set if missing"

. and ...

Written	Read aloud	Example
.	"dot": property or method; chains left to right	<code>a.b.c()</code> , <code>arr.map(f).filter(g)</code>
?.	"if it exists, dot"	<code>a?.b</code>
...x in [] or { }	"spread x in here"	<code>[...a, 1]</code> , <code>{ ...o, k: v }</code>
...x in a parameter list	"collect the rest into x "	<code>(a, ...rest)</code>
...x in destructuring	"and everything else into x "	<code>const { a, ...rest } = o</code>

Brackets

Written	Read aloud	Example
<code>{ }</code> in a value	"an object with ..."	<code>{ a: 1 }</code>
<code>{ }</code> after <code>=></code> , <code>if</code> , <code>for</code> , <code>function</code>	"a block of statements"	<code>() => { return 1 }</code>
<code>{ }</code> in a type	"an object shape with ..."	<code>{ a: number }</code>
<code>{ }</code> in <code>import</code>	"the named export ..."	<code>import { x } from "m"</code>
<code>{ }</code> in destructuring	"pull out keys ..."	<code>const { a } = o</code>
<code>[]</code> in a value	"an array of ..."	<code>[1, 2]</code>
<code>[]</code> after a name	"element at" / "key"	<code>arr[0], obj["k"]</code>
<code>[]</code> after a type	"array of"	<code>number[]</code>
<code>[]</code> in a type with commas	"a tuple of"	<code>[string, number]</code>
<code>[]</code> in destructuring	"pull out elements ..."	<code>const [a, b] = arr</code>
<code>[k]: v</code> in an object literal	"key computed from <code>k</code> "	<code>{ [name]: 1 }</code>
<code>()</code> after a name	"call"	<code>f(x)</code>
<code>()</code> around params	"taking ..."	<code>(a, b) => ...</code>
<code>()</code> around <code>{ }</code> after <code>=></code>	"returns an object"	<code>() => ({ a: 1 })</code>
<code>< ></code> after a name	"of"	<code>Map<string, number> = "a Map of string to number"</code>
<code>< ></code> before <code>(</code> in a declaration	"generic over <code>T</code> "	<code>function f<T>(x: T)</code>
<code>`</code> , <code>`</code>	"a template string"	<code>`Hello \${name}`</code>
<code>\${ }</code> inside backticks	"insert the value of"	<code>`\${a + b}`</code>

`|`, `&`, `||`, `&&`

Written	Where	Read aloud
<code> </code>	in a type	"or"
<code>&</code>	in a type	"and also"
<code> </code>	in a value	"or else" (fallback on falsy)
<code>&&</code>	in a value	"and then" (<code>a && b</code> is <code>b</code> if <code>a</code> is truthy, else <code>a</code>)
<code> </code> , <code>&</code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code>	with numbers	bitwise; rare; read as integer math

Keywords that look like operators

Written	Read aloud	Runtime effect
<code>x as T</code>	" <code>x</code> , which I claim is a <code>T</code> "	none
<code>as const</code>	"freeze these literals as their exact types"	none
<code>satisfies T</code>	"check this matches <code>T</code> , keep its own type"	none
<code>typeof x</code> in a value	"the type name of <code>x</code> , as a string"	yes
<code>typeof x</code> in a type	"the type of <code>x</code> "	none
<code>x instanceof C</code>	"was <code>x</code> built by class <code>C</code> "	yes
<code>"k" in obj</code>	"does <code>obj</code> have key <code>k</code> "	yes
<code>for (k in obj)</code>	"for each key <code>k</code> in <code>obj</code> "	yes
<code>for (x of arr)</code>	"for each value <code>x</code> of <code>arr</code> "	yes
<code>keyof T</code>	"the names of <code>T</code> 's keys"	none
<code>new C()</code>	"build a new <code>C</code> "	yes
<code>await p</code>	"wait for <code>p</code> and unwrap it"	yes
<code>delete o.k</code>	"remove key <code>k</code> from <code>o</code> "	yes
<code>void expr</code>	"run <code>expr</code> , give back undefined"	yes
<code>declare</code>	"this exists somewhere else"	none
<code>readonly</code>	"cannot be reassigned" (type only)	none
<code>#x</code>	"private field <code>x</code> , enforced at runtime"	yes
<code>@decorator</code>	"wrap the next thing with <code>decorator</code> "	yes

Three-second decode

1. Type or value? After a `:` in a declaration, or inside `interface/type`, it is a type and runs nothing. Say “of type ...” for it.
2. Find `=`: say “set [left] to [right]”.
3. Find `=>`: say “[params] returns [right]”.
4. Find `await`: say “wait for ...”; a Promise-returning call with no `await` is where to look for the bug.
5. Find `?`, `!`, `as`: say “the author claims ...” and ask whether the claim holds at runtime.